

01 — Architecture

This document describes how MegaMOO is put together: the object model that the whole world is built from, the path a single command travels from the socket to a running verb and back, and how state is persisted. Read it first — the rest of the manual assumes this vocabulary.

- [The object model](#)
 - [The core object hierarchy](#)
 - [The request lifecycle](#)
 - [The task system](#)
 - [Persistence](#)
 - [Networking and protocols](#)
 - [Concurrency model](#)
-

The object model

Everything in the world — players, rooms, exits, items, even the system object that holds global configuration — is a `M000bject` (`moo/objects.py`). Objects are identified by a stable integer (`objnum`, written `#N`) and arranged in a **single-parent inheritance tree**: each object has exactly one parent and inherits that parent's properties and verbs, LambdaMOO-style.

Native attributes vs. the property system

A `M000bject` distinguishes two kinds of state:

- **Native attributes** are stored directly on the Python instance. The set is fixed (`_NATIVE_ATTRS` in `moo/objects.py`): `objnum`, `children`, `noun`, `aliases`, `owner`, `flags`, `properties`, `verbs`, `created`, `last_move`, `tags`, plus the relationship attributes `parent`, `location`, and `contents` (which have descriptor setters that keep both sides of the relationship in sync).
- **MOO properties** are everything else — `hp`, `description`, `gender`, any property a builder adds with `@adprop` or a verb adds with `add_property()`. These are stored in the object's `properties` dict as `PropertyInfo` records (`moo/objects.py`), each carrying a `value`, an `owner`, and a `perms` string.

The split is invisible at the call site. Attribute access is routed through `M000bject.__getattr__` / `__setattr__`:

- Reading `obj.hp` checks local properties, then walks inheritance (via the cache below), then falls back to a verb of that name (returning a callable), and finally returns a falsy sentinel (`_null_attr`) if nothing matches — so unset properties read as falsy rather than raising.

- Writing `obj.hp = 5` updates the existing property in place or creates a new local property (default perms `rc`), and triggers a write-through to the database when one is attached.

This is why verbs read so cleanly: `pobj.location.contents`, `obj.hp -= damage`, and `getattr(exit, 'jumpable', False)` are all just attribute access, but they transparently respect inheritance, permissions, and persistence.

Property permissions

Each property's `perms` string mixes three flags:

Flag	Meaning
<code>r</code>	Readable by anyone (otherwise only the owner or a wizard)
<code>w</code>	Writable by anyone (otherwise only the owner or a wizard)
<code>c</code>	“Change/inherit” — children inherit a copy of this property

Permission enforcement happens in the verb namespace's safe `getattr / setattr` wrappers (`moov/verb_namespace.py`), not on the raw object — so engine-internal code can do what it needs, while verb code written by a non-wizard programmer is held to the property's declared permissions. See Writing Verbs.

Inheritance, resolved cache, and invalidation

A naive inheritance lookup is $O(\text{depth})$: to read a property you might walk every ancestor. MegaMOO instead builds a **flattened resolution cache** per object — `_resolved_properties` and `_resolved_verbs` — mapping each inherited name to `(definition, defining_objnum)`. Lookups then become a single dict hit regardless of tree depth.

The cache is built lazily on first use (`_ensure_cache`) and **invalidated by cascading down the tree**: when an ancestor's properties or verbs change, the invalidation propagates to all descendants so the next lookup rebuilds. The practical payoff is that you can have deep class hierarchies (base object → item → container → furniture → a specific chair) without paying for the depth on every property read.

Verb resolution (`M000bject.find_verb`) follows the same shape: check local verbs first (so an object can override an inherited verb, and so the minimum abbreviation logic works locally), then consult the resolved-verb cache. Hidden verbs are skipped during player-initiated lookup.

Flags and tags

- **Flags** (`ObjectFlags`) are the classic MOO bits: `PLAYER`, `PROGRAMMER`, `WIZARD`, and so on. Some are kept in sync with the `auth` property — adding `gm3` sets `PROGRAMMER`, `gm4` sets `WIZARD` (see Operations).

- **Tags** are a free-form `category/tag` classification set (managed with `@adtag` / `@rmtag`) used for zones, game-system grouping, and queries.
-

The core object hierarchy

A handful of low-numbered objects form the backbone of any MegaMOO world, and the shipped database defines them out of the box: every object through **#44**, and **#100**, the owner/wizard account and first character in the game. The table below lists the load-bearing ones; they are the conventional layout new content builds against:

#	Object	Parent	Role
#0	System object	—	Holds globals; <code>\$name</code> resolves to <code>#0.name</code>
#1	Root	#0	Ancestor of everything
#2	PlayerObjectDB	#1	Pool of player-account objects
#3	Base_Character	#10	Base for all characters; staff/builder verbs live here . Defines <code>is_char</code> .
#4	OCharacter	#3	Out-of-character (account) character
#5	ICharacter	#3	In-character (in-game) character — the puppetable avatar
#6	Nowhere	#1	Isolation container used during character generation
#7	arch	#22 GoExit	Character-generation entry exit in room #14
#8	Bag	#27	“The Body Bag”; holds the <code>moo_verb_path</code> config
#9	TrashBin	#1	Where recycled objects go
#10	BaseObject	#1	Base for all game objects
#11	object	#10	Generic object; defines the spatial put/get/look hooks
#12	item	#11	Carriable item
#14	(login room)	#16	OOO staging room where new connections arrive (<code>LOGIN_ROOM</code> in <code>moo/globals.py</code>)
#15	BaseRoom	#11	Base room; <code>gmove</code> / <code>match_exit</code>
#16	OCRoom	#15	OOO room; <code>go</code> / <code>look</code>
#17	ICRoom	#15	IC room; <code>go</code> / <code>look</code> plus round-time checks
#20	BaseExit	#11	Base exit; <code>invoke</code> / <code>gmove</code>
#21	DirectionalExit	#20	Virtual compass-direction exits

#	Object	Parent	Role
#22	GoExit	#20	Named, walkable passage exits
#23	ClosableGoExit	#22	Doors/gates that open, close, lock
#24	ClimbableExit	#22	Exits requiring <code>climb</code>
#25	JumpableExit	#22	Exits requiring <code>jump</code>
#26	BaseContainer	#12	Holds items; supports entry/exit
#30	BaseFurniture	#11	Sit/lie targets

Two structural conventions follow from this:

- **Where in-game verbs live.** Player commands are *not* defined on player objects. They live on the room parents — **#16 (OCRoom)** for out-of-character rooms and **#17 (ICRoom)** for in-character rooms — so every room of a type inherits the full command set (`look` , `go` , `get` , `inventory` , and so on). Character objects (#3/#4/#5) carry **only staff verbs**, inherited from #3. Per-object behavior is added not by putting a player verb on the object, but by the room verb calling a `<verb>_` hook on the matched object (e.g. `get` on the room calls `in_get` / `on_get` on the container). Room/player verbs use `pmatch()` and check `obj.existent` after matching; staff verbs on #3 use `bmatch()` . See Writing Verbs.
- **The OOC / IC split** is realized as two character classes: you connect as an OCharacter (#4) in the login room (#14), then *puppet* into an ICharacter (#5) to play. The same player connection drives whichever object is currently puppeted. See [the request lifecycle](#) below and Operations.

The request lifecycle

Here is the full path of a single typed command, e.g. `get 2nd sword from chest` .

1. Network read

A `PlayerConnection` (`moo/network.py`) owns one socket for its whole lifetime. Input is read line by line (capped at 8192 bytes), telnet protocol bytes are stripped/negotiated, and the resulting command string is handed to the parser.

2. Parsing

`CommandParser` (`moo/parser.py`) turns the line into a `ParseResult` :

1. Strip a leading `@` or `+` prefix if present (builder/utility commands).
2. Detect the `/expr` eval shortcut and route it to the `eval` verb.

3. Find the verb by searching, in order, the **player**, the **player's location**, the **other objects in that room**, and finally the **player's inventory** — walking the inheritance chain at each step and stopping at the first match. In practice the player object only contributes staff verbs (from #3); the everyday commands — `look`, `go`, `get`, `inventory` — are found on the room (#16/#17), which is why a player standing in any room has the full command set even though nothing is defined on the player itself.
4. Pull off MUSH-style switches (`look/brief` → verb `look`, switches `['brief']`).
5. Split the remainder into a direct object, a preposition, and an indirect object (`get <dobj> from <iobj>`).

The result carries `verb`, `verb_obj` (where the verb was found), `dobj / dobjstr`, `prep`, `iobj / iobjstr`, `argstr`, `args`, and `switches`.

The parser also *attempts* object resolution, recording object numbers in

`ParseResult.dobj / .iobj`. **Those numbers are then discarded:** the verb type's parse (step 4 below) overwrites the namespace's `dobj / iobj` with plain strings, and the `ParseResult` is consulted only as a fallback if that parse fails. Verbs match their own objects — see Matching objects.

3. Verb lookup

`M000object.find_verb` resolves the matched name against the object that owns it, honoring local overrides, the resolved-verb cache, minimum abbreviations, and the hidden flag. It returns the defining object number and the verb definition — the latter pointing at the `.py` source to execute.

4. Namespace setup and dispatch

The verb does not run inline on the event loop. `execute_command()` builds the namespace and hands the compiled code to a dedicated `ThreadPoolExecutor(max_workers=1)` — the single verb-execution worker — guarded by a 30-second `COMMAND_TIMEOUT`.

A top-level player command does **not** go through the task queue. Nothing in this path creates a `Task`. The queue and `VerbExecutor` (`moo/verbs.py`) serve the *other* entry points — work scheduled by `delay()` / `fork()` — which is also where `TaskLimits` applies. See [the task system](#).

`build_verb_namespace()` (`moo/verb_namespace.py`) constructs the scope the verb code will see: the injected Python builtins, the core context variables (`pobj`, `this`, `db`, `args`, ...), permission-checked `getattr` / `setattr`, the parsed command parts, messaging defaults, and the full MOO builtin library (`pmatch`, `call_verb`, `create`, `move`, `delay`, ..., plus the inherited `obj.msg` / `obj.msg_room` messaging verbs). This is enumerated in Writing Verbs.

The parsed command parts in that namespace are produced by the verb's **verb type** — a pluggable class (default `MasterVerb`) that owns the per-verb `parse()` step, so coders can give a verb a custom argument syntax. See Verb types: customizing the parser.

5. Preprocess, compile, execute

The verb's source is run through `preprocess_verb_code()` (`moo/verbs.py`): `#N` references become `db.get_object(N)`, `$name` references become system-object property access, and the body is wrapped in a function so that `return` works at top level. The result is compiled (and cached) and executed against the prepared namespace. The return value is captured as `namespace['result']`.

If the verb is a **generator** (it used `yield`), the engine starts an interactive session: each `yield "prompt"` sends a prompt to the player and the verb resumes with their next line of input. This is how multi-step flows that prompt the player (such as new-character setup) work mid-verb.

6. Output and persistence

Messaging (`obj.msg`, `obj.msg_room`, `broadcast`) runs the text through emit substitution and the per-connection color/wrap pipeline before writing to the socket. Any property writes the verb made are persisted through the object's write-through to SQLite. The task completes (or errors, or suspends), and the connection loops for the next line.

Connection, login, and puppeting

The lifecycle above assumes a logged-in player. Getting there:

1. A new connection sees a splash screen and the prompt `Enter your username or NEW to create a new account:`.
 2. **Login** (`moo/login.py`) authenticates against an existing account (passwords hashed with `bcrypt` when available, salted SHA-256 otherwise; inline `name password` on one line is supported, as is taking over an existing session). **NEW** runs an account-creation flow (`name` → `password` → `confirm`) that claims a free account object from the player pool (#2).
 3. The connected account object is flagged `PLAYER` and placed in the login room (#14) — the OOC staging area.
 4. From OOC the player **puppets** into an in-character `ICharacter` (#5). Puppeting swaps which object the connection drives and moves the IC character to its stored `last_location`. `exit` unpuppets back to OOC. On disconnect the engine unpuppets cleanly, saving `last_location` and clearing the `PLAYER` flag on the stored account so no “ghost” active flag remains.
-

The task system

Scheduled work runs inside a managed **task** (`moo/tasks.py`) — the queue behind `delay()` and `fork()`:

- **Resource limits** (`TaskLimits`): a tick budget (100,000), a wall-clock budget (5s), a maximum verb-call stack depth (50), and a maximum fork depth (10).
- `delay(seconds, code, context)` schedules a code string to run later without blocking. `fork(seconds, code, context)` does the same for independent follow-up work, with fork-depth limits to prevent fork bombs. `pause(seconds)` is the blocking alternative — it sleeps the shared worker, and is capped at 30s.
- **State machine:** `PENDING` → `RUNNING` → `COMPLETED`, with `SUSPENDED` (used by the scheduler for not-yet-due work) and `ERROR` / `ABORTED` states.

Two caveats worth stating plainly:

- **There is no `suspend()` builtin.** Verb code cannot park itself mid-execution and resume in place; `SUSPENDED` is an internal scheduler state, not an API.
- **`TaskLimits` does not bound top-level player commands**, which bypass the queue (see [step 4](#)). Their guard is the 30-second `COMMAND_TIMEOUT`, plus `MAX_VERB_DEPTH` inside `call_verb`. A verb that spins for 29 seconds will stall every other player, because all verb code shares one worker thread.

See Writing Verbs for the author-facing API.

Persistence

State lives in **SQLite** (`moo/database.py`), not in a flat memory image:

- **Write-through:** property changes on a database-attached object are persisted automatically; verbs do not call an explicit “save” in normal operation.
- **WAL mode** gives crash recovery and lets reads proceed during writes.
- **Lazy LRU object cache:** objects are materialized on demand and evicted under pressure, so a large world does not require holding every object in memory.
- **Checkpointing:** the server periodically writes checkpoint snapshots (interval and retention are configurable; see Operations) into a `<dbname>_checkpoints/` directory beside the database file — `mm.db` checkpoints into `mm_checkpoints/` — pruning old ones.
- **Database creation:** pointing the server at a non-existent database, or giving a template plus an output path, initializes a new database; see Operations.

Verb source is stored twice, deliberately. The authoritative copy at runtime is the `code` column of the `verbs` table — that is what the engine compiles and executes. It is *mirrored* to disk under

`moo verbs/<objnum>/<verbname>.py` , resolved relative to the `moo_verb_path` config stored on #8, so the world's behavior is diff-able and version-controlled.

The two are kept in sync from both directions: `@program` compiles, installs into the live database, and writes the file in one step; and a background watcher (on by default, polling every 2s) re-loads any verb file whose mtime changes on disk, so edits from an external editor, `git pull` , or the MCP bridge go live without a command. `@reload` forces that same disk→database pull on demand. See Hot coding.

Networking and protocols

`moo/network.py` serves Telnet and (optionally) WebSocket from the same asyncio event loop:

- **Protocol negotiation:** the server advertises MXP, GMCP, MSDP, and MSSP, and learns the client's terminal width via NAWS so wrapping matches the real window.
- **MXP clickable links:** text in backticks (``north``) is rendered as a clickable `<send>` link for MXP-capable clients and degrades to plain text otherwise.
- **Color:** `&`-prefixed codes (`&r` red, `&G` bright green, `&<245>` xterm-256 gray, `&<#FF0000>` hex RGB, `&n` reset) are translated to ANSI on output. See Building Worlds.
- **ANSI-aware wrapping:** wrapping measures *visible* width, so escape sequences never count toward the line length or get split. `WRAP_WIDTH` (default 121) can be set to `0` to disable server-side wrapping entirely.
- **Screenreader mode:** the per-player `screenreader` command strips all color and decoration from that player's output stream, persistently.

Limits worth knowing: a cap on simultaneous connections, per-IP connection rate limiting, an idle timeout, an input-line size cap (8192 bytes), and a bounded number of login attempts before disconnect. Exact values live in `moo/globals.py` .

Concurrency model

MegaMOO is **asyncio for I/O, single-threaded for verb execution**. Network connections, timers, and the API are all coroutines on one event loop, but verb code runs on a single one-worker executor so that the object database is only ever mutated by one verb at a time. This sidesteps the entire class of data races that a multi-threaded object store would invite, and it matches the MOO mental model: the world advances one action at a time. Long-running work should be handed to `delay()` / `fork()` and the task scheduler rather than blocking that worker — `pause()` blocks it, and so does any slow verb.

The trade-off — a single CPU core for verb execution — is the right one for a text world, where the work per command is tiny and correctness of shared state matters far more than raw throughput.

Next: Writing Verbs — the namespace, the preprocessor, matching, and messaging, in author-facing detail.